

Bản sơ lược về vài dạng bài ba lô

Xu Chiheng

Trường Trung học Ôn Châu, Chiết Giang

Người hướng dẫn: Shu Chunping

Tháng 12 năm 2008

Nguồn gốc: Bản sơ lược về vài dạng bài ba lô

IOI China candidate team papers / 2009 / Xu Chiheng / knapsack variants

Abstract

Bài toán ba lô là một bài toán kinh điển và là một phần rất cơ bản trong quy hoạch động. Tuy nhiên, lấy bài toán ba lô 0-1 làm gốc, người ta có thể biến đổi ra rất nhiều dạng đề khác nhau; lời giải của chúng tất nhiên cũng không giống nhau, nên rất đáng tiếp tục khảo sát.

Bài viết này thảo luận bốn bài toán biến thể của ba lô và trình bày một số cách làm của tác giả, với mong muốn gợi mở thêm hướng suy nghĩ.

Từ khóa: quy hoạch động, ba lô, tối ưu hóa.

1 Mở đầu

Bài toán ba lô là một bài toán tối ưu kinh điển trong vận trù học. Nó là một bài toán NP-đầy đủ, nhưng lại có bối cảnh ứng dụng thực tế rất rộng, xuất phát từ một tình huống quen thuộc trong đời sống: một chiếc ba lô và rất nhiều vật phẩm; ta cần bỏ một số vật phẩm vào ba lô để đạt một mục tiêu nhất định.

Trong tin học, sau khi lượng hóa mọi dữ liệu, ta thu được bài toán sau.

Bài toán ba lô 0-1. Cho n vật phẩm và một ba lô. Vật phẩm i có giá trị W_i , thể tích V_i , ba lô có dung lượng C . Có thể tùy ý chọn các vật phẩm bỏ vào ba lô; hãy tìm tổng giá trị lớn nhất của các vật phẩm được bỏ vào.

Khi chọn vật phẩm cho vào ba lô, đối với mỗi vật phẩm i , hoặc bỏ nó vào, hoặc không bỏ. Không được bỏ vật phẩm i nhiều lần, cũng không được chỉ bỏ một phần của vật phẩm i (tức là không được chia vật phẩm). Vì vậy bài toán này được gọi là bài toán ba lô 0-1.

Các phương pháp thường dùng để giải ba lô 0-1 gồm quay lui, tham lam, thuật toán di truyền, tìm kiếm tabu, mô phỏng luyện kim, v.v.

Ở bậc phổ thông, cái gọi là bài toán ba lô 0-1 kinh điển thường đảm bảo mọi dữ liệu sau khi lượng hóa đều là số nguyên dương, tức là một trường hợp đặc biệt của quy hoạch nguyên. Trong bài viết này, nếu không nói khác, ta đều dùng giả thiết đó. Lời giải quy hoạch động kinh điển có độ phức tạp $\mathcal{O}(nC)$ như sau.

Trạng thái: xét i vật phẩm đầu tiên, chọn một số vật phẩm sao cho tổng thể tích không vượt quá j , giá trị lớn nhất đạt được là $F_i[j]$. Quyết định hiện tại là vật phẩm thứ i có được chọn hay không. Ta có chuyển tiếp:

$$F_i[j] = F_{i-1}[j] \quad (V_i > j \geq 0),$$

$$F_i[j] = \max\{F_{i-1}[j], F_{i-1}[j - V_i] + W_i\} \quad (C \geq j \geq V_i).$$

Do F_i chỉ liên quan tới F_{i-1} , có thể dùng mảng cuộn để tiết kiệm bộ nhớ. Pseudocode kinh điển là:

```
FOR i := 1 TO n
  FOR j := C DOWNTO Vi
    Max(F[j], F[j - Vi] + Wi) -> F[j]
  END FOR
END FOR
```

2 Các biến đổi cơ bản của ba lô

2.1 Ba lô hoàn toàn

Bài toán ba lô hoàn toàn. Cho n loại vật phẩm và một ba lô. Loại vật phẩm thứ i có giá trị W_i , thể tích V_i , ba lô có dung lượng C , và số lượng mỗi loại vật phẩm là vô hạn. Có thể tùy ý chọn các vật phẩm bỏ vào ba lô; hãy tìm tổng giá trị lớn nhất của các vật phẩm được bỏ vào.

Bài này hoàn toàn có thể chuyển thành ba lô 0-1: chia loại vật phẩm thứ i thành $\lfloor C/V_i \rfloor$ vật phẩm riêng biệt, rồi áp dụng quy hoạch động kinh điển của ba lô 0-1. Tuy nhiên độ phức tạp thời gian của cách này quá cao, không phải một cách cài đặt thực dụng.

Để thấy bài toán này là một cực khác so với ba lô 0-1: mỗi loại vật phẩm đều có thể lấy không giới hạn. Chỉ cần đổi phương trình chuyển tiếp là có thể xây dựng thuật toán mới.

Trạng thái: xét i loại vật phẩm đầu tiên, chọn một số vật phẩm sao cho tổng thể tích không vượt quá j , giá trị lớn nhất đạt được là $F_i[j]$. Quyết định hiện tại là có chọn loại vật phẩm thứ i hay không, và nếu chọn thì có thể chọn nhiều cái. Chuyển tiếp là:

$$F_i[j] = F_{i-1}[j] \quad (V_i > j \geq 0),$$

$$F_i[j] = \max\{F_{i-1}[j], F_{i-1}[j - V_i] + W_i\} \quad (C \geq j \geq V_i).$$

Chú ý hạng thứ hai là F_i , không phải F_{i-1} . Điểm khác ba lô 0-1 chỉ nằm ở đây, vì ta được phép thêm một vật phẩm nữa trên cơ sở đã chọn trước đó.

Như vậy bài toán có lời giải $\mathcal{O}(nC)$, cùng bậc với ba lô 0-1, và cũng có thể dùng mảng cuộn:

```
FOR i := 1 TO n
  FOR j := Vi TO C
    Max(F[j], F[j - Vi] + Wi) -> F[j]
  END FOR
END FOR
```

2.2 Ba lô nhiều lần

Bài toán ba lô nhiều lần. Cho n loại vật phẩm và một ba lô. Loại vật phẩm thứ i có giá trị W_i , thể tích V_i , số lượng K_i , ba lô có dung lượng C . Có thể tùy ý chọn các vật phẩm bỏ vào ba lô; hãy tìm tổng giá trị lớn nhất của các vật phẩm được bỏ vào.

Giống ba lô hoàn toàn, bài này cũng có thể trực tiếp dùng quy hoạch động kinh điển của ba lô 0-1, nhưng hiệu quả quá thấp, nên cần tìm thuật toán hiệu quả hơn.

Trước hết, với loại vật phẩm thứ i , ta không thể xác định ngay lấy bao nhiêu cái là tối ưu, vì không có gì chứng minh rằng lấy một cái hoặc lấy hết sẽ nhất định tốt hơn. Nói cách khác, số lượng cần trong nghiệm tối ưu có thể là bất cứ số nào từ 0 đến K_i .

Trong đời sống, nếu cần có khả năng đưa ra bất kỳ số tiền nguyên nào từ 1 đến K_i , ta thường không mang K_i tờ một đơn vị, vì quá bất tiện; thay vào đó ta mang vài tờ mệnh giá 1 và một số tờ có mệnh giá khác nhau. Ý tưởng này có thể dùng trực tiếp trong bài toán.

Không cần chia một loại vật phẩm thành K_i vật phẩm đơn lẻ; chỉ cần chia nó thành các nhóm sao cho có thể ghép được mọi số lượng từ 1 đến K_i . Có thể chứng minh rằng cách tách nhĩ phân làm số nhóm nhỏ nhất: tách K_i thành

$$1, 2, 4, \dots, 2^t, K_i - 2^{t+1} + 1 \quad (2^{t+2} > K_i \geq 2^{t+1}).$$

Như vậy đã đủ đáp ứng mọi số lượng có thể xuất hiện trong nghiệm tối ưu. Sau đó vẫn dùng thuật toán kinh điển của ba lô 0-1. Ta thu được thuật toán có độ phức tạp

$$\mathcal{O}\left(C \sum_i \lfloor \log_2 K_i \rfloor\right).$$

```
FOR i := 1 TO n
  1 -> m
  WHILE Ki > 0
    IF m > Ki THEN Ki -> m
    Ki - m -> Ki
    FOR j := C DOWNT0 Vi * m
      Max(F[j], F[j - Vi * m] + Wi * m) -> F[j]
    END FOR
    m * 2 -> m
  WEND
END FOR
```

Thuật toán này còn có một tối ưu: nếu $K_i > C/V_i$, thì phần vượt quá là vô nghĩa và có thể bỏ đi. Độ phức tạp khi đó có thể viết là $\mathcal{O}(nC \log_2 C)$. Vì trong các bài ở bậc phổ thông, giá trị C thường có giới hạn vừa phải, thuật toán này trong thực tế đã đủ đáp ứng nhu cầu thông thường.

Tuy vậy, phần sau có một cách giải hiệu quả hơn.

2.3 Tối ưu bằng hàng đợi đơn điệu

Bài toán tổng đoạn liên tiếp lớn nhất có giới hạn độ dài. Cho dãy X_i dài n . Hãy tìm tổng lớn nhất của một đoạn liên tiếp có độ dài không vượt quá $l_{\max}$.

Trước hết xét cách đơn giản nhất: dùng hai vòng lặp $\mathcal{O}(nl_{\max})$ để tìm giá trị lớn nhất.

```
FOR i := 1 TO n
  0 -> s
  FOR j := i DOWNT0 Max(i - lmax + 1, 1)
    s + Xj -> s
    IF s > ans THEN s -> ans
  END FOR
END FOR
```

Gọi S_i là tổng từ X_1 đến X_i . Nếu đã cố định một đầu mút, việc còn lại là lấy một cực trị trên một đoạn liên tiếp của mảng S . Bài toán cực trị đoạn có thể giải bằng cây đoạn. Nhưng bài này còn có tính chất khác: độ dài đoạn là cố định theo cửa sổ trượt, và mỗi đoạn chỉ cần truy vấn một lần. Vì vậy có thể dùng cấu trúc đơn giản hơn: hàng đợi đơn điệu.

Xét hàng đợi đơn điệu để duy trì giá trị lớn nhất. Sau khi bảo đảm chỉ số trong dãy tăng dần, ta còn yêu cầu giá trị phần tử giảm dần:

vị trí trong hàng đợi	L	$L + 1$	$\dots$	$R - 1$	R
chỉ số	A_L	A_{L+1}	$\dots$	A_{R-1}	A_R
giá trị	B_L	B_{L+1}	$\dots$	B_{R-1}	B_R

Thỏa mãn $A_{i+1} > A_i > A_{i-1}$ và $B_{i-1} > B_i > B_{i+1}$ với $R > i > L$.

Ngoài thao tác lấy phần tử đầu hàng đợi, hàng đợi đơn điệu cần thêm thao tác để duy trì tính chất đặc biệt của nó. Nếu đưa vào một phần tử mới (a, b) , thì a chắc chắn lớn hơn A_R , nhưng b có thể lớn hơn hoặc bằng B_R . Vì $b \geq B_R$, trong khi phần tử R sẽ rời hàng đợi trước phần tử mới, nên phần tử R đã mất giá trị sử dụng: từ nay về sau phần tử mới chắc chắn không kém phần tử R . Do đó ta có thể xóa phần tử R .

Lặp lại bước trên cho đến khi phía trước không còn phần tử nào hoặc thỏa mãn $B_R > b$, rồi mới cho phần tử mới vào hàng đợi. Rõ ràng phần tử đầu hàng đợi hiện tại chính là giá trị lớn nhất của đoạn.

```

PROCEDURE INSERT a, b
  WHILE R >= L AND b > B[R] DO R - 1 -> R
  R + 1 -> R
  a -> A[R]
  b -> B[R]
END

```

Hàng đợi đơn điệu như vậy không bảo đảm mỗi thao tác riêng lẻ đều là $\mathcal{O}(1)$, nhưng vì mỗi phần tử chỉ vào hàng đợi một lần và ra hàng đợi một lần, tổng hiệu quả là $\mathcal{O}(n)$.

Dĩ nhiên, bài toán vừa nêu chỉ là chức năng cơ bản của hàng đợi đơn điệu. Điều ta cần là dùng nó để tối ưu ba lô, nên ta xét lại bài toán ba lô nhiều lần.

Với loại vật phẩm thứ i , biết thể tích v , giá trị w , số lượng k . Có thể chia toàn bộ mảng quy hoạch động thành v phần theo phần dư của thể tích j modulo v . Ví dụ với $v = 3$:

j	0	1	2	3	4	5	6	7	8	$\dots$
$j \bmod v$	0	1	2	0	1	2	0	1	2	$\dots$

Ta xử lý riêng từng phần. Giả sử phần dư là d :

chỉ số j	0	1	2	3	4	5	$\dots$
thể tích tương ứng	d	$d + v$	$d + 2v$	$d + 3v$	$d + 4v$	$d + 5v$	$\dots$

Sau khi chia nhóm, trạng thái có chỉ số j có thể chuyển từ bất kỳ chỉ số nào trong đoạn $j - k$ đến $j - 1$, vì hai thể tích kề nhau đúng bằng v . Điều này đã khá giống bài toán cực trị đoạn. Nhưng cần chú ý rằng do thể tích khác nhau, giá trị ở thể tích lớn hiển nhiên có thể lớn hơn hoặc bằng thể tích nhỏ; so sánh trực tiếp là vô nghĩa. Vì vậy cần hiệu chỉnh giá trị về cùng một mốc thể tích. Chẳng hạn cùng quy về d , tức là đưa

$$F[jv + d] - jw$$

vào hàng đợi thay cho giá trị ban đầu.

Với vật phẩm thứ i , pseudocode chuyển tiếp là:

```

FOR d := 0 TO v - 1                // duyệt phần dư, xử lý riêng
  xóa hàng d
  FOR j := 0 TO (C - d) div v      // j là chỉ số, thể tích là j*v+d
    INSERT j, F[j * v + d] - j * w
    IF A[L] < j - k THEN L + 1 -> L
    B[L] + j * w -> F[j * v + d]
  END FOR
END FOR

```

Vì hàng đợi đơn điệu có tổng độ phức tạp tuyến tính, ba lô nhiều lần sau tối ưu bằng hàng đợi đơn điệu có độ phức tạp $\mathcal{O}(nC)$.

3 Một vài dạng ba lô khác

3.1 Ba lô phụ thuộc dạng cây: chọn môn

Bài toán ba lô phụ thuộc dạng cây. Cho n vật phẩm và một ba lô. Vật phẩm thứ i có giá trị W_i , thể tích V_i , nhưng phụ thuộc vào vật phẩm thứ X_i : chỉ sau khi chọn X_i mới được chọn i ; nếu không có phụ thuộc thì $X_i = 0$. Quan hệ phụ thuộc tạo thành một rừng. Ba lô có dung lượng C . Có thể tùy ý chọn vật phẩm bỏ vào ba lô; hãy tìm tổng giá trị lớn nhất của các vật phẩm được bỏ vào.

Bài này cần dùng ba lô trên nền tảng quy hoạch động trên cây.

Vật phẩm tổng quát

Xét một loại “vật phẩm” không có chi phí (thể tích) và giá trị cố định; thay vào đó, giá trị của nó thay đổi theo chi phí (thể tích) mà ta phân bổ cho nó.

Vật phẩm tổng quát có thể biểu diễn bằng một mảng một chiều mô tả quan hệ giữa thể tích và giá trị: G_j là giá trị tương ứng khi thể tích là j , với $C \geq j \geq 0$. Rõ ràng mảng quy hoạch động ba lô F_i trước đây chính là một vật phẩm tổng quát, vì $F_i[j]$ biểu diễn giá trị lớn nhất khi thể tích là j . Tương tự, nhiều vật phẩm cũng có thể gộp thành một vật phẩm tổng quát.

Tổng của vật phẩm tổng quát

Phép gộp hai vật phẩm tổng quát thành một vật phẩm tổng quát chính là việc liệt kê cách phân bổ thể tích cho hai vật phẩm, thỏa mãn:

$$G[j] = \max_{0 \leq k \leq j} \{G_1[j - k] + G_2[k]\} \quad (C \geq j \geq 0).$$

Thời gian gộp hai vật phẩm tổng quát là $\mathcal{O}(C^2)$.

Với bài toán ba lô có quan hệ phụ thuộc dạng cây, có thể xem mỗi cây con là một vật phẩm tổng quát. Khi đó vật phẩm tổng quát của một cây con bằng tổng của vật phẩm tổng quát ứng với gốc cây con và các vật phẩm tổng quát ứng với mọi cây con nối với gốc.

Toàn bộ quá trình quy hoạch động đi từ lá lên gốc. Với mỗi cây con, thao tác là:

```

PROCEDURE DEAL i, v, w           // nút i, thể tích v, giá trị w
  FOR j := v TO C               // khởi tạo vật phẩm tổng quát này
    w -> Fi[j]
  END FOR

```

```

END FOR
FOR s := 1 TO n
  IF s là con của i THEN
    tong của Fi va Fs -> Fi
  END IF
END FOR
END

```

Mỗi lần chỉ gộp được hai vật phẩm tổng quát. Muốn gộp n vật phẩm tổng quát thành một vật phẩm cần $n - 1$ lần gộp, nên thuật toán có hiệu quả $\mathcal{O}(nC^2 + n^2)$. Phần $\mathcal{O}(n^2)$ tất nhiên có thể dùng danh sách kê để giảm thành $\mathcal{O}(n)$, do đó độ phức tạp cuối cùng là $\mathcal{O}(nC^2)$.

Nhìn lại ba lô 0-1 kinh điển: trong thuật toán đó, tổng của một vật phẩm tổng quát và một vật phẩm thường chỉ cần $\mathcal{O}(C)$. Từ đó suy ra:

Tổng của vật phẩm tổng quát và một vật phẩm thường

Gộp một vật phẩm tổng quát với một vật phẩm thường thành một vật phẩm tổng quát có thể làm bằng cách tương tự quy hoạch động kinh điển của ba lô 0-1:

$$G[j] = G_1[j] \quad (v > j \geq 0),$$

$$G[j] = \max\{G_1[j], G_1[j - v] + w\} \quad (C \geq j \geq v).$$

Việc gộp như vậy chỉ tốn $\mathcal{O}(C)$, cũng là gộp thêm một vật phẩm nhưng nhanh hơn rất nhiều so với tổng của hai vật phẩm tổng quát. Vậy có cách nào dùng kiểu gộp $\mathcal{O}(C)$ này thay cho việc tính tổng của hai vật phẩm tổng quát để xử lý bài toán hay không?

Hợp của vật phẩm tổng quát

Vì giữa hai vật phẩm tổng quát có phần giao, nên không thể đồng thời chọn cả hai. Khi đó ta cần lấy **hợp** của vật phẩm tổng quát: với cùng một thể tích, chọn cái có giá trị lớn hơn, độ phức tạp $\mathcal{O}(C)$:

$$G[j] = \max\{G_1[j], G_2[j]\} \quad (C \geq j \geq 0).$$

Xét lại việc xử lý cây con gốc i . Giả sử hiện cần xử lý một con s của i . Nếu trong F_i hiện tại ta bắt buộc đưa vật phẩm s vào làm trạng thái ban đầu của cây con gốc s , thì sau khi xử lý xong cây con gốc s , F_s là một vật phẩm tổng quát có giao với F_i (thực tế là F_s chứa F_i). Đồng thời, F_s phải thỏa mãn đã chọn vật phẩm s , nên $F_s[j]$ với $V_s > j \geq 0$ đã vô nghĩa, còn $F_s[j]$ với $C \geq j \geq V_s$ chắc chắn chứa vật phẩm s . Để tiện cài đặt, sau khi xử lý, chương trình quy định chỉ $F_s[j]$ với $C - V_s \geq j \geq 0$ là hợp lệ.

Tiếp theo chỉ cần lấy hợp của F_s và F_i rồi gán cho F_i , là hoàn tất việc xử lý một con. Như vậy tổng độ phức tạp chỉ là $\mathcal{O}(nC)$.

```

PROCEDURE DEAL i, C
  FOR s := 1 TO n
    IF s là con của i THEN
      Fi -> Fs
      DEAL s, C - Vs
      FOR k := Vs TO C
        Max(Fi[k], Fs[k - Vs] + Ws) -> Fi[k]
      END FOR
    END IF
  END FOR
END

```

```

        END FOR
    END IF
END FOR
END

```

Dùng thuật toán này có thể tối ưu các bài kiểu “Chọn môn” xuống $\mathcal{O}(nC)$. Nó cũng có thể dùng làm một lời giải quy hoạch động trên cây $\mathcal{O}(nC)$ cho bài “Kế hoạch ngân sách của Jinming” của NOIP 2006. Sau khi bài viết hoàn thành, tác giả được TKY nhắc rằng thuật toán cùng độ phức tạp đã được đề cập trong bài năm 2007 “Bàn về tổ chức dữ liệu hợp lý” của He Sen; vì vậy cách làm ở đây chỉ nên xem là một cài đặt $\mathcal{O}(nC)$ đơn giản hơn.

3.2 PKU 3093

PKU 3093. Cho n vật phẩm và một ba lô. Vật phẩm thứ i có thể tích V_i , ba lô có dung lượng C . Yêu cầu chọn một số vật phẩm bỏ vào ba lô sao cho mọi vật phẩm còn lại đều không thể bỏ thêm vào được; hãy tính số phương án.

Tạm thời chưa xét điều kiện “mọi vật phẩm còn lại đều không thể bỏ thêm vào được”. Khi đó bài toán chỉ là đếm mọi phương án khả thi của ba lô 0-1.

Dùng $F_i[j]$ biểu diễn số phương án chọn một số vật phẩm trong i vật phẩm đầu tiên sao cho tổng thể tích bằng j . Khởi tạo $F_0[0] = 1$. Chuyển tiếp:

$$F_i[j] = F_{i-1}[j] \quad (V_i > j),$$

$$F_i[j] = F_{i-1}[j] + F_{i-1}[j - V_i] \quad (j \geq V_i).$$

Rõ ràng thuật toán này có hiệu quả $\mathcal{O}(nC)$, và nó tính số phương án của mọi cách đặt vật phẩm vào ba lô.

Bây giờ xét điều kiện “mọi vật phẩm còn lại đều không thể bỏ thêm vào”. Nếu vật phẩm nhỏ nhất trong các vật phẩm còn lại có thể tích v , thì số phương án là

$$\sum_{C \geq j > C-v} F_n[j],$$

với điều kiện ta đã xác định trước vật phẩm nhỏ nhất còn lại là vật phẩm nào.

Sau khi sắp xếp theo thể tích, bước tiếp theo là liệt kê i làm một vật phẩm nhỏ nhất trong phần còn lại. Với mọi vật phẩm $s < i$, bắt buộc phải bỏ vào ba lô; với i , không được bỏ vào; với mọi vật phẩm $s > i$, làm thống kê phương án khả thi của ba lô 0-1, rồi cộng $\sum F_n[j]$ với $C \geq j > C - V_i$ vào *ans*. Vì mỗi lần cần thống kê trên $n - i$ vật phẩm và tổng cộng thống kê n lần, độ phức tạp là $\mathcal{O}(n^2C)$.

```

0 -> sum                                // sum la tong the tich vat pham 1..i-1
FOR i := 1 TO N
    xoa mang F
    1 -> F[sum]                            // khoi tao
    FOR s := i + 1 TO N                    // thong ke phuong an kha thi
        FOR j := C DOWNTO Vs + sum
            F[j] + F[j - Vs] -> F[j]
        END FOR
    END FOR
    FOR k := C DOWNTO C - Vi + 1 // cong vao tong so phuong an
        IF k >= sum THEN ans + F[k] -> ans
    END FOR

```

```

    sum + Vi -> sum
END FOR

```

Có thể thấy cùng một vật phẩm được xét đưa vào ba lô nhiều lần, gây lãng phí thời gian. Quan sát thấy vật phẩm thứ i được xét tổng cộng $i - 1$ lần. Mỗi vòng lặp lại bớt đi một vật phẩm. Nếu đảo ngược toàn bộ quá trình, liệu mỗi vật phẩm có thể chỉ xét một lần không?

Do trạng thái khởi tạo khác nhau, trước hết cần thống nhất trạng thái khởi tạo. Có thể để mỗi lần $F[0] = 1$, tổng dung lượng là $C - sum$. Nhưng chỉ thống nhất khởi tạo vẫn chưa đủ, vì dung lượng ba lô mỗi lần vẫn khác nhau. Khi làm thống kê ba lô, dung lượng không được là một biến thay đổi, mà cũng phải thống nhất; vì vậy mỗi lần xét một vật phẩm đều dùng dung lượng lớn nhất C để cập nhật ba lô.

Sau một thao tác, cộng

$$\sum F[j] \quad (C - sum \geq j > C - sum - V_i, j \geq 0)$$

vào *ans*. Khi đó mỗi vật phẩm chỉ được xét một lần, dung lượng ba lô thống nhất là C , và độ phức tạp trở thành $\mathcal{O}(nC)$.

```

0 -> sum
1 -> F[0]
FOR i := 1 TO n
    sum + Vi -> sum
END FOR
FOR i := n DOWNT0 1
    sum - Vi -> sum
    FOR j := C - sum DOWNT0 Max(C - sum - Vi + 1, 0)
        ans + F[j] -> ans
    END FOR
    FOR j := C DOWNT0 Vi
        F[j] + F[j - Vi] -> F[j]
    END FOR
END FOR

```

4 Tổng kết

Nhìn lại bốn bài toán ba lô trong bài viết: với ba lô hoàn toàn, chỉ cần đổi phương trình chuyển tiếp; với ba lô nhiều lần, dùng tối ưu hàng đợi đơn điệu để đạt $\mathcal{O}(nC)$; trong ba lô phụ thuộc dạng cây, ta đưa vào khái niệm mới và cuối cùng chuyển hóa thuật toán; còn ở bài PKU 3093, ta hợp nhất các thao tác tương tự để tối ưu.

Mặc dù các phương pháp được dùng không giống nhau, mỗi phương pháp đều không chỉ giới hạn trong bài toán ba lô, mà hoàn toàn có thể vận dụng linh hoạt ở những bài toán khác.

Mọi bài được nhắc tới trong bài đều được giải bằng thuật toán thời gian $\mathcal{O}(nC)$. Điều này không có nghĩa mọi bài ba lô đều có thể tối ưu đến mức đó, nhưng có thể khẳng định rằng không thể có độ phức tạp nhanh hơn tuyến tính theo kích thước bằng nC nếu vẫn phải xử lý toàn bộ bề mặt trạng thái này.

Hiện nay vẫn còn rất nhiều bài toán dạng ba lô chưa có lời giải thật tốt, chờ mọi người tiếp tục tìm tòi và nghiên cứu.

A Tài liệu tham khảo

- *Chín bài giảng về ba lô*, DDengi, ZJU.

B Các đề bài gốc được nhắc trong bài

B.1 Chọn môn

Nguồn: CTSC 1997.

Ở đại học áp dụng chế độ tín chỉ. Mỗi môn học có một số tín chỉ nhất định; chỉ cần sinh viên chọn môn đó và qua kiểm tra thì có thể nhận số tín chỉ tương ứng. Tổng tín chỉ cuối cùng của sinh viên là tổng tín chỉ của các môn đã chọn.

Mỗi sinh viên đều phải chọn một số lượng môn quy định. Một số môn có thể chọn trực tiếp, còn một số môn cần kiến thức nền tảng nhất định: chỉ sau khi đã chọn một số môn khác mới được chọn. Ví dụ, “Cấu trúc dữ liệu” phải được chọn sau “Lập trình ngôn ngữ bậc cao”; khi đó ta gọi môn sau là môn tiên quyết của môn trước. Mỗi môn có nhiều nhất một môn tiên quyết trực tiếp. Hai môn học cũng có thể có cùng một môn tiên quyết. Để tiện mô tả, mỗi môn có một mã số liên tiếp 1, 2, 3, Ví dụ:

mã môn	môn tiên quyết	tín chỉ
1	không có	1
2	1	1
3	2	3
4	không có	3
5	2	4

Trong ví dụ trên, 1 là môn tiên quyết của 2; nếu muốn chọn 2 thì nhất định 1 đã được chọn. Tương tự, nếu muốn chọn 3, thì 1 và 2 đều phải đã được chọn.

Sinh viên không thể học hết mọi môn mà trường đại học mở, nên phải chọn trước những môn mình muốn học khi nhập học. Tổng số môn mỗi sinh viên được chọn là cho trước. Hãy tìm một phương án chọn môn sao cho nhận được nhiều tín chỉ nhất và phải thỏa mãn nguyên tắc môn tiên quyết đi trước. Giả sử giữa các môn không có xung đột thời gian.

Input. Dòng đầu của tệp vào gồm hai số nguyên dương M, N cách nhau bởi một dấu cách; M là tổng số môn chờ chọn ($1 \leq M \leq 1000$), N là tổng số môn sinh viên có thể chọn ($1 \leq N \leq M$).

Tiếp theo là M dòng, mỗi dòng mô tả một môn, mã môn lần lượt là 1, 2, ..., M . Mỗi dòng có hai số: số đầu là mã môn tiên quyết của môn này (0 nếu không có), số thứ hai là tín chỉ. Tín chỉ là số nguyên dương không vượt quá 10.

Output. Dòng đầu chỉ có một số, là tổng tín chỉ của các môn thực sự được chọn. Tiếp theo là N dòng, mỗi dòng một số, biểu diễn mã môn được chọn.

Ví dụ.

INPUT.TXT

7 4

2 2

0 1

```
0 4
2 1
7 1
7 6
2 2
```

OUTPUT.TXT

```
13
2
6
7
3
```

Mã nguồn của bài “Chọn môn”.

```
var
  n,C,i:longint;
  x,w:array[1..400]of longint;
  f:array[0..400,0..400]of longint;

procedure dfs(k,C:longint);
var
  i,j:longint;
begin
  if C<=0 then exit;
  for i:=1 to n do if x[i]=k then begin
    for j:=0 to C-1 do f[i,j]:=f[k,j]+w[i]; {bat buoc chon i}
    dfs(i,C-1);
    for j:=1 to C do
      if f[i,j-1]>f[k,j] then
        f[k,j]:=f[i,j-1]; {lay hop}
    end;
  end;
end;

begin
  readln(n,C);
  for i:=1 to n do begin
    readln(x[i],w[i]); {cha x[i] va tin chi w[i]}
  end;
  dfs(0,C); {0 la cha cua moi nut khong co cha}
  writeln(f[0,C]);
end.
```

B.2 Kế hoạch ngân sách của Jinming

Nguồn: NOIP 2006, bài 2.

Mô tả bài toán. Jinming rất vui vì căn nhà mới của gia đình sắp được nhận chìa khóa; trong nhà mới có một căn phòng rộng rãi dành riêng cho cậu. Điều làm cậu vui hơn là hôm qua mẹ nói: “Phòng của con cần mua những đồ gì, bố trí thế nào thì con tự quyết, miễn là không vượt quá N đồng.” Sáng nay Jinming bắt đầu lập ngân sách. Cậu chia các món muốn mua thành hai loại: vật chính và phụ kiện. Phụ kiện thuộc về một vật chính nào đó, ví dụ:

vật chính	phụ kiện
máy tính	máy in, máy quét
tủ sách	sách
bàn học	đèn bàn, văn phòng phẩm
ghế làm việc	không có

Nếu muốn mua một vật được phân loại là phụ kiện, trước hết phải mua vật chính mà phụ kiện đó thuộc về. Mỗi vật chính có 0, 1, hoặc 2 phụ kiện. Phụ kiện không có phụ kiện của riêng nó. Jinming muốn mua rất nhiều thứ, chắc chắn vượt quá giới hạn N đồng của mẹ. Vì vậy cậu quy định độ quan trọng của từng món theo 5 mức, dùng số nguyên 1 đến 5, trong đó mức 5 là quan trọng nhất. Cậu cũng tra trên Internet giá của từng món (đều là bội của 10 đồng). Cậu muốn trong điều kiện không vượt quá N đồng, làm cho tổng tích giữa giá tiền và độ quan trọng của mỗi món được mua là lớn nhất.

Giả sử món thứ j có giá $v[j]$, độ quan trọng $w[j]$, và đã chọn k món với chỉ số $j_1, j_2, \dots, j_k$. Đại lượng cần tối đa là

$$v[j_1]w[j_1] + v[j_2]w[j_2] + \dots + v[j_k]w[j_k].$$

Hãy giúp Jinming thiết kế một danh sách mua sắm thỏa mãn yêu cầu.

Input. Dòng đầu của tệp `budget.in` gồm hai số nguyên dương cách nhau bởi dấu cách:

$$N \quad m.$$

Trong đó $N < 32000$ là tổng tiền, $m < 60$ là số món muốn mua. Từ dòng thứ hai đến dòng $m + 1$, dòng thứ j cho dữ liệu cơ bản của món có số hiệu $j - 1$, gồm ba số nguyên không âm:

$$v \quad p \quad q.$$

Trong đó $v < 10000$ là giá, $p \in \{1, \dots, 5\}$ là độ quan trọng, và q cho biết vật này là vật chính hay phụ kiện: nếu $q = 0$, đây là vật chính; nếu $q > 0$, đây là phụ kiện, và q là số hiệu của vật chính mà nó thuộc về.

Output. Tệp `budget.out` chỉ có một số nguyên dương, là giá trị lớn nhất không vượt quá tổng tiền, nhỏ hơn 200000.

Ví dụ.

Input
1000 5
800 2 0
400 5 1
300 5 1
400 3 0
500 2 0

Output
2200

Chương trình trong bài gốc.

```
var
  n,C,i:longint;
  x,w,v:array[1..60]of longint;
  f:array[0..60,0..3200]of longint;

procedure dfs(k,C:longint);
var i,j:longint;
begin
  if C<=0 then exit;
  for i:=1 to n do if x[i]=k then begin
    for j:=0 to C-v[i] do f[i,j]:=f[k,j]+w[i]; {bat buoc chon i}
    dfs(i,C-v[i]);
    for j:=v[i] to C do
      if f[i,j-v[i]]>f[k,j] then f[k,j]:=f[i,j-v[i]]; {lay hop}
    end;
  end;
end;

begin
  assign(input,'budget.in'); reset(input);
  assign(output,'budget.out'); rewrite(output);
  readln(C,n);
  C:=C div 10;
  for i:=1 to n do begin
    readln(v[i],w[i],x[i]); {chi phi, do quan trong, cha}
    w[i]:=w[i]*v[i];
    v[i]:=v[i] div 10;
  end;
  dfs(0,C); {0 la cha cua moi nut khong co cha}
  writeln(f[0,C]);
  close(output);
end.
```

B.3 Margaritas on the River Walk

Nguồn: PKU 3093, Greater New York 2006.

Mô tả. Một hoạt động phổ biến ở San Antonio là thưởng thức margarita trong công viên dọc con sông, khu vực được gọi là River Walk. Có thể mua margarita tại nhiều cơ sở dọc River Walk, từ các khách sạn sang trọng đến quầy Joe's Taco and Margarita. Giá margarita thay đổi tùy theo lượng và chất lượng nguyên liệu cũng như không gian của cơ sở. Bạn dành ra một khoản tiền nhất định để thử nhiều loại margarita khác nhau.

Cho giá của một ly margarita tại mỗi cơ sở và số tiền được phân bổ, hãy tính có bao nhiêu tổ hợp cực đại khác nhau có thể mua, với điều kiện chọn nhiều nhất một ly từ mỗi cơ sở. Một tổ hợp hợp lệ phải có tổng giá không vượt quá số tiền được phân bổ, và số tiền chưa dùng phải nhỏ hơn giá của mọi cơ sở chưa được chọn. Nếu không, ta vẫn còn có thể thêm cơ sở đó vào tổ hợp.

Ví dụ, giả sử có 25 đô-la và các giá nguyên đô-la là:

Vendor	A	B	C	D	H	J
Price	8	9	8	7	16	5

Các tổ hợp có thể là:

$ABC(25), ABD(24), ABJ(22), ACD(23), ACJ(21),$
 $ADJ(20), AH(24), BCD(24), BCJ(22), BDJ(21),$
 $BH(25), CDJ(20), CH(24), DH(23), HJ(21).$

Vậy tổng số tổ hợp là 15.

Input. Dữ liệu bắt đầu bằng một dòng chứa số nguyên N , là số bộ dữ liệu tiếp theo ($1 \leq N \leq 1000$). Mỗi bộ dữ liệu bắt đầu bằng một dòng chứa hai số nguyên V và D , lần lượt là số cơ sở ($1 \leq V \leq 30$) và số đô-la được phép chi ($1 \leq D \leq 1000$). Hai số cách nhau bởi một hoặc nhiều dấu cách. Phần còn lại của mỗi bộ dữ liệu gồm một hoặc nhiều dòng, mỗi dòng chứa một hoặc nhiều số nguyên là giá margarita của các cơ sở. Tổng cộng có V giá được cho. Giá của một ly margarita luôn ít nhất là 1. Dữ liệu được chọn sao cho kết quả nằm trong số nguyên không dấu 32 bit.

Output. Với mỗi bộ dữ liệu, in một dòng gồm số thứ tự bộ dữ liệu, một dấu cách, rồi số tổ hợp của bộ dữ liệu đó.

Ví dụ.

Sample Input

```
2
6 25
8 9 8 7 16 5
30 250
1 2 3 4 5 6 7 8 9 10 11
12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
```

Sample Output

```
1 15
2 16509438
```

Gợi ý. Một số phương pháp giải có thể có độ phức tạp lũy thừa theo số cơ sở. Với các phương pháp này, giới hạn thời gian có thể bị vượt ở những bộ dữ liệu có số cơ sở lớn, chẳng hạn ví dụ thứ hai ở trên.